Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу

«Операционные системы»

Группа: М8О-212БВ-25

Студент: Штыков М.С.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 13.10.2026

Москва, 2026

Постановка задачи

Вариант 1

Разработать две отдельные программы на языке C для обмена данными между родительским и дочерним процессами через каналы. Пользователь вводит имя выходного файла, затем строки целых чисел типа int. Родитель передаёт строки ребёнку; ребёнок вычисляет сумму чисел каждой строки и записывает её в файл. Количество чисел произвольно. Необходимо обрабатывать системные ошибки.

Общий метод и алгоритм решения

Среда выполнения: Ubuntu/Linux, язык C99. Ввод-вывод реализован через системные интерфейсы POSIX; stdio.h не используется.

1. Родитель читает имя файла и создаёт два канала: pipe1 для передачи чисел ребёнку, pipe2 для возврата эха родителю.

2. После fork() в дочерней ветке dup2() подключает pipe1 к стандартному вводу, pipe2 — к стандартному выводу. execv() запускает отдельную программу child с именем файла в argv[1]. Неиспользуемые концы каналов закрываются.

3. Ребёнок открывает файл и последовательно разбирает числа. По символу перевода строки записывает сумму и обнуляет состояние. Полученный блок возвращает родителю как эхо.

4. Пустая строка завершает ввод. Родитель закрывает пишущий конец pipe1; ребёнок получает EOF, закрывает файл и завершается. Родитель получает его статус через waitpid().

Использованные системные интерфейсы

pipe — создание каналов; fork — создание процесса; dup2 — перенаправление дескрипторов; execv — запуск программы ребёнка; open, read, write, close — работа с файлами и каналами; waitpid — ожидание ребёнка; readlink — определение пути к программе; sigaction — настройка SIGPIPE; _exit — завершение при ошибке запуска.

Особенности реализации

Канал передаёт поток байтов: один read может вернуть часть строки или несколько строк. Состояние разбора сохраняется между чтениями, поэтому буфер 4096 байтов не ограничивает число слагаемых. Входные значения проверяются на диапазон int, сумма хранится в long long с контролем переполнения. write_all учитывает частичные записи, receive_echo дочитывает весь ответ. Некорректный ввод завершает программу с ошибкой; ранее записанные суммы сохраняются.

Код программы

Ключевые фрагменты реализации. Вспомогательные функции и часть проверок ниже не приведены.

parent.c

```
pid = fork();
switch (pid) {
case -1:
   report_error("fork");
   goto cleanup;
case 0: {
   if (close_fd(&pipe1[1]) < 0 || close_fd(&pipe2[0]) < 0) _exit(1);
   if (dup2(pipe1[0], STDIN_FILENO) < 0 ||
      dup2(pipe2[1], STDOUT_FILENO) < 0) {
      report_error("dup2"); _exit(1);
   }
   if (close_fd(&pipe1[0]) < 0 || close_fd(&pipe2[1]) < 0) _exit(1);
   char *const args[] = {"child", filename, NULL};
   execv(child_path, args);
   report_error("exec child");
   _exit(1);
}

default:
   break; /* Ниже работает только родитель. */
}
```

В родительском цикле передача блока и получение эха:

```
write_all(pipe1[1], buf, count);
receive_echo(pipe2[0], count);
```

child.c

```
int file = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC | O_APPEND, 0600);
Parser parser = {0};
    for (ssize_t i = 0; i < bytes; ++i) {
      if (buf[i] == '\n') {
         if (save_sum(file, &parser) < 0) goto cleanup;
      } else consume(&parser, buf[i]);
    }

    if (write_all(STDOUT_FILENO, buf, (size_t)bytes) < 0) {
      report_error("write pipe2"); goto cleanup;
    }
```

save_sum завершает последнее число, проверяет команду и записывает сумму через put_number; затем состояние Parser обнуляется.

Протокол работы программы

Сборка на Ubuntu компилятором GCC в стандарте C99:

```
gcc -std=c99 -Wall -Wextra -Wpedantic parent.c -o parent
gcc -std=c99 -Wall -Wextra -Wpedantic child.c -o child
```

Тест 1. Несколько команд и завершение пустой строкой

После запуска ./parent вводится имя result.txt, затем строки 1 2 3, 10 -5 7 и 100. Ещё одно нажатие Enter на пустой строке завершает работу. Родитель выводит эхо введённых команд.

sshty@Sht:**/mnt/c/Users/sshty/Desktop/OS 2026/lab1**$ gcc parent.c -Wall -Wpedantic -std=c99 -Wextra -Werror -o parent
sshty@Sht:**/mnt/c/Users/sshty/Desktop/OS 2026/lab1**$ gcc child.c -Wall -Wpedantic -std=c99 -Wextra -Werror -o child
sshty@Sht:**/mnt/c/Users/sshty/Desktop/OS 2026/lab1**$ ./parent
Output filename: reuslt.txt
Enter integers, one command per line. Empty line = finish.
1 2 3
1 2 3
10 -10 0
10 -10 0
777773747 -17777177 170493
777773747 -17777177 170493

sshty@Sht:**/mnt/c/Users/sshty/Desktop/OS 2026/lab1**$ cat reuslt.txt
6
0
760167063

Тест 2. Диапазон чисел и тип суммы

sshty@Sht:**/mnt/c/Users/sshty/Desktop/OS 2026/lab1**$ ./parent
Output filename: result.txt
Enter integers, one command per line. Empty line = finish.
-2147483648 2147483647 1
-2147483648 2147483647 1
sshty@Sht:**/mnt/c/Users/sshty/Desktop/OS 2026/lab1**$ cat result.txt
0

sshty@Sht:**/mnt/c/Users/sshty/Desktop/OS 2026/lab1**$ ./parent
Output filename: result.txt
Enter integers, one command per line. Empty line = finish.
2147483647 2147483647
2147483647 2147483647

sshty@Sht:**/mnt/c/Users/sshty/Desktop/OS 2026/lab1**$ cat result.txt
4294967294

Оба случая обработаны корректно: сумма может превышать INT_MAX.

Тест 3. Некорректный ввод

sshty@Sht:**/mnt/c/Users/sshty/Desktop/OS 2026/lab1**$ ./parent
Output filename: result.txt
Enter integers, one command per line. Empty line = finish.
abc + 1
Invalid integer, empty command or sum overflow.
Child closed its output pipe.
Child did not finish successfully.

Дополнительные проверки

Проверены строка из 20000 единиц (сумма 20000), разрыв числа на границе буфера, несколько строк за одно чтение и EOF без перевода строки. Проверены ошибки открытия и записи файла, а также отсутствие исполняемого child. Результаты соответствуют ожидаемым.

Вывод

Реализован обмен между двумя процессами через неименованные каналы. Освоены создание процесса, перенаправление дескрипторов и запуск отдельной программы через execv. При реализации учтены частичная передача данных, разрыв строк между чтениями и закрытие каналов для получения EOF. Проверки подтвердили корректный подсчёт сумм и обработку ошибочных ситуаций.